AI Engineer / FDE Interview — Spoken Answers

Oct 27, 2026 · @Aditya

How to use these answers

Every answer follows the same four beats, so the interviewer can follow your thinking: **frame the problem, lay out the options with their costs, give a recommendation, prove it with evidence.** Each script runs about 350 to 550 words, which is 2.5 to 4 minutes at a calm 140 words a minute.

- **Frame first.** Senior answers start by naming what to measure or ask, not by listing tricks.
- **Every lever has a price.** Say the cost out loud: quality, latency, money, or engineering effort.
- **Close on evals.** No change ships without a regression gate. This line alone signals seniority.
- **Anchor lines** (marked *Your anchor*) tie the answer to the RTB Fallout Agent. Use them only as stated: pilot at enterprise clients, a single agentic system, gpt-4o-mini behind the gateway.

Numbers in the worked examples are illustrative and approximate. Say "roughly" or "in the order of" when you use them.

Q1 · LLM latency: five changes and what they cost

About 3.5 minutes.

Before I change anything, I'd split "slow" into its parts, because each part has a different fix. There's **time to first token**, which is mostly prompt processing, or prefill, plus anything that runs before the model, like retrieval and tool calls. Then there's **time per output token**, because decoding is sequential: a 1,000-token answer is 1,000 forward passes. And there's the **number of round trips**, which matters a lot in agents. So I trace a few hundred real requests and look at p50 and p95 for each stage. Very often the answer is boring: the prompt grew to 15,000 tokens, or the agent now makes six sequential calls.

With that breakdown, these are the five changes I'd consider.

1. **Generate fewer output tokens.** Output tokens are usually the biggest single driver of end-to-end latency. I'd ask for structured output instead of prose, set tight max-token limits, and remove "explain your reasoning" where nobody reads it. *Cost:* answers become terser, you risk truncation, and some tasks lose accuracy when you remove reasoning, so this needs an eval.
2. **Shrink and cache the prompt.** Put the static parts first, meaning the system prompt, tool definitions and few-shot examples, so prefix caching can skip recomputing them. Cut retrieved context from, say, 20 chunks to 5 with a reranker. *Cost:* prompt-layout discipline, an extra reranker hop of tens of milliseconds, and a recall risk if you trim too hard.
3. **Use a smaller or faster model for the steps that allow it.** Classification, extraction and routing rarely need the frontier model. If we self-host, speculative decoding and quantisation also help. *Cost:* quality risk on harder inputs, which means a router and evals to hold a quality floor.
4. **Parallelise and stream.** Run independent retrievals and tool calls concurrently, collapse sequential agent steps where one call can do the job, and stream tokens to the user so perceived latency drops even when total time doesn't. *Cost:* more complex orchestration, and streaming makes output guardrails harder, because you are checking a partial answer.
5. **Infrastructure and caching.** Cache full responses for repeated questions, co-locate the app in the same region as the model endpoint, and for steady traffic buy provisioned or dedicated throughput so you are not queuing behind other tenants. *Cost:* real money for reserved capacity, plus staleness and correctness risk with response caching.

My recommendation is to go in that order: the first two are nearly free and often give a 2x improvement on their own. The model change is the most powerful but also the riskiest. Whatever we change, it goes through the same eval suite, with latency and quality gates side by side, because a fast wrong answer is not a win.

Your anchor: in the RTB Fallout Agent, the eval layer carries cost and latency gates in CI/CD next to quality checks, so a latency fix that degrades recommendations cannot ship.

Q2 · Prompting vs RAG vs fine-tuning: when the customer wants the model to "know" their data

About 3 minutes.

When a customer says "I want the model to know our data", the first thing I do is unpack the word *know*, because it means three different things. It can mean **facts**: our policies, our product catalogue, last week's tickets. It can mean **behaviour**: answer in our format, use our terminology, call our tools the right way. Or it can mean a **skill**: reason about our domain the way a senior analyst does. The rule of thumb is that **retrieval is for facts, and fine-tuning is for behaviour.** Fine-tuning is a poor way to inject facts. The model learns them unreliably, can't cite them, can't forget them on request, and still hallucinates around them with full confidence.

An analogy I like: RAG is an open-book exam, and fine-tuning is training a new employee's habits. You don't train someone to memorise the policy manual; you train them how to work, and you hand them the manual.

So I walk a ladder, cheapest first.

1. **Prompting.** If the knowledge is small and stable, put it in the prompt. With today's long context windows and prompt caching, a few hundred pages of static reference material can simply live in the prompt. That's the fastest thing to ship and the easiest to change.
2. **RAG.** As soon as the data is bigger than the context, changes often, has per-user permissions, or needs citations, I move to retrieval. Freshness is just re-indexing, access control is a filter at query time, and the user gets a source link to verify. *Cost:* an ingestion pipeline, an index to run, and a new failure mode, because if retrieval misses, the answer is wrong.
3. **Fine-tuning.** I reach for it when the failure is behavioural and consistent: wrong output format despite good prompts, domain jargon misread, tool-calling style that's off. Or when I want a small model to do a narrow task at a fraction of the cost. *Cost:* you need hundreds to thousands of quality examples, a training and eval pipeline, and you inherit a maintenance bill, because when the base model is deprecated, you retrain.

The questions I ask the customer map straight onto this ladder: How big is the data? How often does it change? Who is allowed to see what? Do users need citations? Do we have labelled examples of good answers? What latency and cost budget do we have?

In practice, mature systems combine them: RAG for the facts, a light fine-tune or strong prompt for the behaviour. But I always start with prompting and RAG, because they're reversible, and I only fine-tune once evals show a behavioural gap that prompting can't close.

Q3 · PII protection: no PII to an external LLM API

About 3.5 minutes.

I'd start by pinning down what the constraint actually is, because it changes the architecture. What counts as PII here: names and phone numbers only, or also account IDs and free-text notes? Is it a legal requirement, say GDPR or India's DPDP Act, or an internal policy? And what does "external" mean: outside the company's cloud tenancy, or outside its legal entity? A model hosted by a hyperscaler inside the customer's own cloud account, with zero data retention and a private endpoint, satisfies some of those definitions and not others.

Then I offer four architectures, from lightest to strongest.

1. **Redact and re-hydrate at a gateway.** Every call passes through an LLM gateway inside the customer's boundary. It detects PII with a mix of NER models, regex and customer dictionaries, swaps each value for a reversible token like `PERSON_1` or `ACCOUNT_3`, keeps the mapping in a vault, sends the sanitised prompt out, and swaps the real values back into the response. *Cost:* detection is never 100%, so you need a measured recall target; and some meaning is lost, because a token can't tell the model a name's gender or a city's region.
2. **Bring the model inside the boundary.** Use the provider's model through the customer's own cloud account and region, with private networking and no retention. *Cost:* contracts and region availability, and a strict reading of the policy may still say no.
3. **Self-host an open-weight model** in the customer's VPC or data centre. Nothing leaves. *Cost:* GPUs, an inference platform, an on-call team, and usually some quality gap versus frontier models.
4. **Split the workflow.** A small local model handles every step that touches PII: extraction, classification, redaction. The external frontier model only gets the de-identified reasoning task. This is often the sweet spot: frontier quality where it matters, full control where the data is.

Whatever pattern we choose, a few controls are non-negotiable. The gateway must be the **only** egress path, enforced at the network level, so no team can bypass it. The system must **fail closed**: if the PII detector errors, the call is blocked, not sent. And we must look at the **side channels**, which is where leaks really happen: prompts copied into observability and tracing tools, logs, the vector store, and the embeddings API. Calling an external embedding service is also sending data out.

Finally, I'd prove it rather than promise it: a labelled test set of documents with known PII, a measured redaction recall reported to the security team, and audit logs of every outbound call.

My default recommendation is the gateway plus the split workflow, because it gives the customer both a hard guarantee and frontier quality, and it degrades gracefully into full self-hosting if the policy tightens.

Q4 · Hallucinations: why they happen, three fixes and their costs

About 3 minutes.

The root cause is the training objective. An LLM is trained to produce the most **plausible** next token, not the most **true** one. Its knowledge sits in its weights as a lossy compression of the training data, so for rare facts, the plausible-sounding reconstruction and the real fact drift apart. On top of that, training and benchmarks have historically rewarded a confident guess over "I don't know", much like a multiple-choice exam with no penalty for wrong answers. So the model learns to guess fluently.

In enterprise systems there's a second source: the model may be fine, but **retrieval handed it the wrong context**, or the model ignored the right context. It helps to separate two kinds: *intrinsic* hallucinations that contradict the source, and *extrinsic* ones that add claims the source doesn't support. You fix them differently, so you measure them separately.

Three techniques I'd use, with their costs:

1. **Ground the answer and allow abstention.** Retrieve the relevant sources, instruct the model to answer only from them, require a citation for every claim, and explicitly allow "I don't have enough information". *Cost:* retrieval infrastructure and latency. And it moves the risk rather than removing it: if retrieval brings the wrong chunk, the model answers confidently from the wrong source. So you must evaluate retrieval separately.
2. **Verify after generating.** Add a check layer: a groundedness judge, or a natural-language-inference model, that tests each claim against the cited source; or sample the answer several times and treat disagreement as low confidence. *Cost:* typically 2 to 5 times the tokens and added latency, and the judge has its own error rate, so it must be calibrated against human labels.
3. **Constrain what the model is allowed to produce.** Use structured outputs, enumerations and tool calls for anything factual. Don't let the model recall a balance or compute a date; make it call the database or the calculator. *Cost:* engineering effort

and less flexibility, but this removes whole classes of error rather than reducing their rate.

Two things I'd say don't work as well as people hope: lowering the temperature reduces variety, not falsehood; and "don't hallucinate" in the prompt does very little.

The design principle I hold is that **the recovery ladder must end in abstention or escalation, never a confident wrong answer.**

Your anchor: in the RTB Fallout Agent, the model never writes to billing directly. It emits a structured JSON of the exact fields to fix; the billing service applies its own rules check; and complex cases route to a human. That's technique three plus human escalation.

Q5 · KV caching and prefix caching

About 3.5 minutes.

Start with how a transformer generates text. To produce each new token, every layer runs attention: the new token's **query** vector is compared against the **key** vectors of every earlier token, and the result is a weighted mix of their **value** vectors. The keys and values for earlier tokens never change once computed. So without a cache, generating token 1,000 would recompute keys and values for the 999 tokens before it, and token 1,001 would do it all again. That's wasted, quadratic work.

KV caching fixes that within one request. We store the key and value tensors for every token, at every layer, in GPU memory. Each new step computes the query, key and value only for the newest token, appends its key and value to the cache, and attends over the cache. This splits inference into two phases. **Prefill** processes the whole prompt in parallel and builds the cache; it is compute-bound and drives time to first token. **Decode** produces one token at a time by reading the cache; it is memory-bandwidth-bound and drives time per output token.

The catch is memory. The cache size per token is:

```
\text{KV bytes per token} = 2 \times n_{\text{layers}} \times n_{\text{kv heads}} \times d_{\text{head}} \times \text{bytes per value}
```

The 2 is for keys and values. For an 8-billion-parameter model with 32 layers, 8 KV heads, head dimension 128 and 16-bit values, that's 2 × 32 × 8 × 128 × 2, which is 128 kilobytes per token. An 8,000-token conversation needs about 1 gigabyte of cache, per user. That's why

techniques like grouped-query attention, which shares key-value heads, and paged attention in vLLM, which allocates the cache in blocks like virtual memory, exist: GPU memory for the cache limits how many users you can batch.

Prefix caching extends the idea *across* requests. Many requests begin with an identical prefix: the same system prompt, tool definitions, few-shot examples, or the same long document. If the KV cache for that prefix is already computed, a new request can reuse it and skip prefill for those tokens. vLLM does this by hashing cache blocks; SGLang uses a radix tree of prefixes; API providers expose it as prompt caching, and bill cached input tokens at a steep discount.

Why it cuts latency: prefill of a 10,000-token prefix is skipped, so time to first token drops sharply, and so does cost. What it does **not** cut is decode time per output token.

The practical rule: the match must be **exact, from the very first token**. Change one token early and everything after it is a cache miss. So you put static content first and dynamic content last, and you never put a timestamp or request ID at the top of the system prompt.

An analogy: the KV cache is the notes you take while reading a chapter, so you don't reread it for every sentence you write. Prefix caching is photocopying those notes for every student who reads the same chapter.

Q6 · LLM cost: getting to 10x cheaper without losing quality

About 3.5 minutes.

The first thing I'd say is that you almost never get 10x from one lever. You get it by **multiplying four or five modest levers**, and you can only find them once you can see where the money goes. LLM cost is simply: number of calls, times tokens per call, times price per token. Each lever attacks one of those three terms.

So step zero is **attribution**. Tag every call with feature, tenant, agent step and model, and build a cost dashboard. In almost every engagement, a small number of call types carry most of the spend, and there is plain waste: retry storms, agents looping eight times, a 12,000-token prompt where 3,000 would do, or a frontier model doing classification.

Then I work through the levers, roughly in order of risk:

1. **Don't make the call.** Exact and semantic response caching for repeated questions, de-duplication, and replacing LLM calls with plain code where the task is deterministic. Capping agent loops and fixing the tools that cause retries also lives here.

2. **Send fewer tokens.** Trim the prompt, rerank to fewer chunks, summarise long conversation history, cap output length. Then lay the prompt out for **prefix caching**, because cached input tokens are billed at a small fraction of the normal price.
3. **Use a cheaper model for most traffic.** A router or cascade sends the easy majority to a model that can be 10 to 20 times cheaper per token. For a high-volume narrow task, distil: fine-tune a small model on the frontier model's best outputs.
4. **Use a cheaper pricing mode.** Anything non-interactive, such as nightly summaries or backfills, goes to batch APIs, which are typically around half price. Steady, predictable volume can justify committed capacity or self-hosting.

Here's how they multiply, with illustrative numbers. Removing waste gives about 1.5x. Trimming tokens plus prefix caching gives about 2x. Routing 70% of traffic to a model 15 times cheaper takes cost from 100 to about 35, so nearly 3x. Batch and caching add maybe 1.2x. Multiply them: 1.5 × 2 × 2.9 × 1.2 is a little over 10x.

Preserving quality is the half of the question that matters most. Every lever is a hypothesis tested on the eval suite before rollout: shadow the new path on live traffic, compare per slice, then canary with a quality floor, for example "no slice drops more than 2 points". Cost is tracked as a first-class metric next to quality and latency, so a later prompt change can't quietly double the spend.

The way I'd frame it to a customer: cost reduction is an engineering programme with a dashboard and a gate, not a one-off model swap.

Q7 · An eval suite that catches silent regressions after a model upgrade

About 3.5 minutes.

The word that matters in this question is **silent**. A model upgrade rarely fails loudly. The average score often goes up, while something specific quietly breaks: the JSON gains an extra field, refusals rise on one topic, answers get 40% longer, or tool calls pass an argument in a slightly different format. So the suite has to be designed to find **changes in behaviour on slices**, not just a drop in an average.

I'd build it in five parts.

1. **A golden dataset drawn from production.** Sample real traces, stratified by intent, customer segment and difficulty, so rare but important slices are represented. Add every past incident as a permanent test case, plus adversarial and edge cases. Version

the dataset like code. A few hundred to a couple of thousand cases is usually enough to start.

2. **Tiered graders, cheapest first.** Deterministic checks catch the most dangerous silent breaks: schema validity, required fields, correct tool and arguments, exact-match answers, length limits. Then reference-based checks against known good answers. Then **LLM-as-judge** with a written rubric for things like helpfulness and groundedness. The judge is itself calibrated against human labels, so we know how far to trust it.
3. **Pairwise comparison.** For open-ended outputs, I ask the judge "which is better, old or new?" rather than scoring each in isolation. Pairwise comparison is far more sensitive to small shifts than absolute scores.
4. **Slice-level reporting with statistics.** Report every slice separately, run each case a few times because outputs are non-deterministic, and put confidence intervals on the differences, so we don't chase noise or miss a real 3-point drop.
5. **Behavioural fingerprints.** Track output-length distribution, refusal rate, tool-call rate, latency and cost per case. These catch the "everything passes but feels different" class of regression.

The process around it matters as much as the suite. **Pin model versions** to dated snapshots and never use floating aliases in production. Keep a calendar of deprecation dates so upgrades are planned, not forced. **Pin the judge model too**, otherwise your measuring stick moves along with the thing you're measuring. Run the suite in CI on any change to the model, prompt, retrieval or tools. Then shadow traffic, then a canary with automatic rollback, then online monitoring of production signals such as escalation rate and user feedback.

One expectation I set with customers: a new model often needs its prompts re-tuned. So the upgrade plan includes prompt work, and the eval suite is what tells you when you're done.

Your anchor: in the RTB Fallout Agent, the eval layer runs golden-path checks, LLM-as-judge, and cost and latency gates in CI/CD.

Q8 · Multi-model routing: cutting cost while holding a quality floor

About 3.5 minutes.

I'd start by defining the **quality floor** concretely, because "hold quality" means nothing until it's a number on an eval set. For example: on every slice of the golden set, the routed system must score at least 97% of what the frontier model alone scores, and never fail a

hard check like schema validity. Everything else is tuning to meet that number at the lowest cost.

There are two basic designs.

A **router** decides up front which model handles a request. It uses cheap signals: task type from an intent classifier, input length, domain, customer tier, or a small trained classifier that predicts "will the small model get this right?". You train that classifier on your own eval data, by running both models and labelling where the small one was good enough.

A **cascade** tries the cheap model first and escalates only if the answer fails a check. The check can be schema validity, a tool call that errors, low confidence from token log-probabilities, disagreement across a few samples, or a small judge model.

The cost maths makes the trade-off clear. Say the small model costs 1 unit and the large one 15. A cascade costs 1, plus the escalation rate times 15. At a 30% escalation rate, that's 5.5 units instead of 15, so about 2.7x cheaper. A router sending 70% to the small model costs 0.7 + 4.5 = 5.2 units, similar money, but with different risks.

- The **cascade** is safer, because every answer is checked, but escalated requests pay for both models and take the latency of both.
- The **router** is faster, but a misrouted hard question just gets a worse answer, and nobody notices unless you measure it.

So in practice I often build a hybrid: route the obvious cases directly, the trivially easy to the small model and the clearly hard to the big one, and cascade only the ambiguous middle.

Holding the floor over time needs three things. **Calibrate the thresholds** on the eval set so the floor is met with some margin. **Monitor online**: escalation rate, per-slice quality from sampled judging, and user signals, because input distributions drift and a router tuned in March is wrong by June. And **maintain prompts per model**, since one prompt rarely suits both.

I'd put the router inside the LLM gateway, which gives a bonus: the same mechanism provides failover when a provider has an outage, and it keeps the application independent of any single model.

Your anchor: in the RTB Fallout Agent, the LLM gateway already does routing, failover and token budgets, so routing policy lives in the platform, not in agent code.

Q9 · Self-hosting an open-weight model vs paying an API provider

About 3.5 minutes.

I frame this as a decision on five axes, and I ask the customer about each before I quote any numbers.

1. **Data and sovereignty.** Is there a hard rule that data cannot leave their boundary? If yes, that can decide the question on its own.
2. **Quality needed.** For the hardest reasoning and agentic tasks, frontier API models usually still lead. For narrow, well-defined tasks like extraction, classification or summarisation, a fine-tuned open model often matches them.
3. **Volume and traffic shape.** Steady, high, predictable volume favours self-hosting. Spiky or low volume favours paying per token.
4. **Control.** Owning the weights means no forced deprecations, a fixed version you can audit, full freedom to fine-tune, and control over latency.
5. **Team capability.** Self-hosting is running a production GPU platform: an inference engine like vLLM or TensorRT-LLM, autoscaling, monitoring, security patching, model upgrades and an on-call rotation.

The number that decides the economics is **GPU utilisation**. Here's an illustrative calculation. Say a GPU rents for about 3 dollars an hour, and serves a small model at 2,000 output tokens per second with good batching. That's 7.2 million tokens an hour, so roughly 40 cents per million tokens at full load. Very competitive. But at 20% utilisation, which is common for a single product with daytime traffic, the same GPU costs about 2 dollars per million tokens. And we haven't yet counted the engineers. A GPU sitting idle overnight is the most expensive token generator you can buy.

So the honest total cost of ownership includes GPUs or reserved instances, redundancy across zones, the platform team's time, evaluation and upgrade work, and the opportunity cost of not getting the next frontier model on day one.

I also remind customers it isn't a binary choice. There's a middle ground: **managed open-weight endpoints**, **dedicated or provisioned throughput** from a provider, and **proprietary models hosted inside their own cloud account**. These solve many sovereignty concerns without running a GPU fleet.

My usual recommendation follows the product's maturity. **Start on APIs** to find product fit fast and to build the eval suite. Once a workload is steady, narrow and high-volume, **move that workload** to a fine-tuned small open model, self-hosted or managed. **Keep the**

frontier API for the hard tail. And put everything behind an LLM gateway with an OpenAI-compatible interface, so moving a workload between providers is a configuration change, not a rewrite.

The one-liner I'd leave them with: self-hosting is cheaper per token only when you keep the GPUs busy and you're ready to run a platform. Otherwise you're paying for the privilege of being on call.

Q10 · Scaling RAG from 50 documents to 5 million

About 4 minutes.

A 50-document proof of concept hides almost every hard problem. With 50 documents, any retriever finds the right one, nobody has permissions, nothing changes, and you can eyeball every answer. At 5 million, five things break: **ingestion, retrieval quality, access control, index cost, and your ability to know whether it works.** I'd address each.

First, ingestion becomes a data platform. Connectors to the source systems, with incremental sync or change data capture rather than full reloads. Robust parsing, including tables, scanned PDFs needing OCR, and slides. De-duplication, both exact hashing and near-duplicate detection, because large corpora are full of copies and old versions. Metadata extraction: owner, date, product, document type, and the access control list. All of it queue-based, idempotent and retryable. And a plan for re-embedding: when you change the embedding model, you build a new index alongside and switch over, blue-green style.

Second, retrieval quality. With millions of chunks, near-misses crowd out the right answer. So I'd use structure-aware chunking that respects headings, with parent-child retrieval: match on small chunks, return the larger parent for context. Then hybrid search, BM25 keyword search plus dense vectors, fused with reciprocal rank fusion, because keywords catch exact IDs and codes that embeddings miss. Then metadata filters to narrow the search space, query rewriting for vague questions, and a cross-encoder reranker over the top 50 or so.

Third, access control. Permissions are enforced at query time as a filter **inside** the search, synced from the source systems. Never retrieve the top 10 and then filter, because that both leaks information and silently destroys recall when 8 of the 10 are forbidden.

Fourth, the index itself. Some quick sizing: 5 million documents at roughly 20 chunks each is 100 million vectors. At 1,024 dimensions in 32-bit floats, that's about 400 gigabytes of raw vectors. That pushes you to quantisation, int8 cuts it 4x and binary far more, plus disk-

based indexes like DiskANN or IVF with product quantisation, plus sharding and replicas for throughput.

Fifth, evaluation and observability. Build a labelled query set and measure retrieval on its own, with recall at k and nDCG, separately from answer quality. Track freshness, the lag between a source change and the index, as a service-level objective. Log which chunks were used, so every bad answer can be debugged.

One more point that shows maturity: not every question is a retrieval question. "How many contracts expire next quarter?" is an aggregation, and no top-k search answers it. At this scale I'd add a query router that sends structured questions to SQL or an API tool, and keeps RAG for genuine document lookup.

Your anchor: the RTB Fallout Agent's retrieval already uses this pattern at its scale: hybrid vector KNN plus BM25 fused with RRF, then an LLM rerank, with separate knowledge namespaces weighted by precedence.

Follow-up probes to practise

Interviewers rarely stop at the first answer. Answer each of these out loud in under a minute, without looking above.

- [ ] **Q1:** Time to first token is fine, but the total answer takes 20 seconds. Which two of your five levers matter, and which don't?
- [ ] **Q2:** The customer insists on fine-tuning on their 10,000 policy documents. How do you push back, and what would you agree to?
- [ ] **Q3:** Your redaction gateway has 98% recall. Is that good enough? What do you do about the other 2%?
- [ ] **Q4:** Retrieval returned the right chunk and the model still contradicted it. Name two causes and a fix for each.
- [ ] **Q5:** A 70B model has 80 layers, 8 KV heads, head dimension 128, 16-bit values. What is the KV cache for one 32,000-token request?
- [ ] **Q6:** The CFO wants the 10x in one quarter. Which levers can ship in two weeks, and which need the eval suite first?
- [ ] **Q7:** Your LLM judge says the new model is 4% better, but users complain. What went wrong in the suite?
- [ ] **Q8:** Escalation in your cascade climbs from 25% to 60% in a month. What happened, and what do you check first?

- [ ] **Q9:** Traffic is 90% during business hours in India. How does that change your self-hosting maths?
- [ ] **Q10:** A user asks "what changed in our refund policy since 2023?" Why does plain top-k retrieval fail, and what do you add?

Answer key for Q5: 2 × 80 × 8 × 128 × 2 bytes is 320 KB per token; times 32,000 tokens is about 10 GB for a single request.